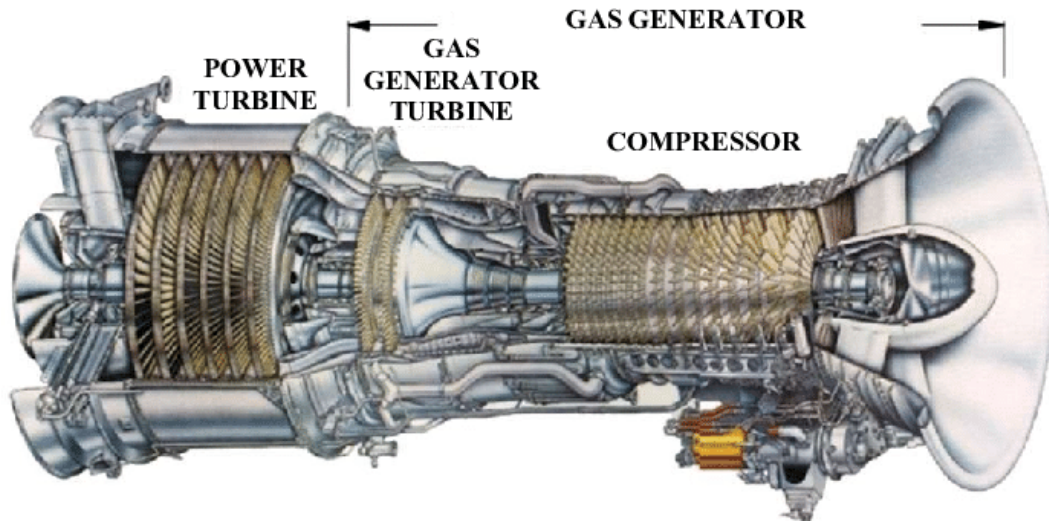


DEPARTMENT OF ICT AND NATURAL SCIENCES & DEPARTMENT OF
OCEAN OPERATIONS AND CIVIL ENGINEERING

AIS4004 - DIGITAL TWINS FOR PREDICTIVE MAINTENANCE

NORWEGIAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Marine Vessel Propulsion Systems



Author:
Mathias, 10040 (Marius), Vetle and 10029 (William)

Table of Contents

List of Figures	iii
List of Tables	iii
1 Introduction	1
2 Theory	2
2.1 Machine Learning Concepts	2
2.1.1 Supervised Learning	2
2.2 Unsupervised learning	2
2.2.1 Reinforcement Learning	2
2.3 Semi-supervised and self-supervised learning	2
2.4 Machine learning models	2
2.4.1 Support vector machine	3
2.4.2 Random forest	3
2.4.3 Extreme gradient boosting	4
2.5 Large language models	4
2.6 Dashboard framework	5
2.7 Correlation analysis	5
2.8 Performance metrics	5
2.8.1 Mean absolute error	6
2.8.2 Coefficient of determination	6
3 Methodology	7
3.1 Data description	7
3.2 Data preparation	7
3.3 Model training and configuration	8
3.4 Model evaluation approach	9
4 Digital Twin Implementation	10
4.1 Digital Twin System Architecture	10
4.2 Model Prediction	11
4.3 Health Monitoring	11
4.4 Maintenance Event Detection	11
4.5 Possible Root Cause Indication	12
4.6 Remaining Useful Life Estimation	12
4.7 Dashboard Implementation	13
4.8 AI-Agent Purpose and Design	13
4.9 AI-Agent Inputs and Outputs	13
4.10 Prompt Structure	14
4.11 Example AI-Agent Response	14
5 AI-Agent Prediction	15
5.1 AI-Agent Purpose and design rationale	15
5.2 AI System Architecture	15
5.3 Context assembly	15
5.4 Agent capabilities	15
5.5 Prompt engineering	16
5.6 Integration with dashboard	16
6 Results and Discussion	17
6.1 Data exploration results	17
6.2 Model comparison	18
6.3 Digital twin demonstration	20
6.4 AI-agent demonstration	21
7 Conclusion	24

Bibliography	25
Appendix	26

List of Figures

1	Illustration of the Support Vector Machine principle.	3
2	Illustration of the Random Forest principle.	4
3	Illustration of the XGBoost principle.	4
4	Data preparation workflow before model training.	8
5	Digital twin system architecture. Sensor data flows through preprocessing and machine learning prediction into the digital twin, which performs health monitoring, maintenance detection, diagnostic indication, and RUL estimation before presenting the results in the Gradio dashboard and AI-agent interface.	10
6	Turbine torque and fuel flow rate at selected speed states.	17
7	Correlation matrix for the propulsion dataset.	18
8	Prediction performance for compressor and turbine decay coefficients.	19
9	Digital twin dashboard during normal operation. Both compressor and gas turbine are above the warning threshold and marked as healthy.	20
10	Digital twin dashboard with an injected turbine fault. The gas turbine health index falls below the warning threshold and is marked as critical.	20
11	Digital twin dashboard at another mission point, showing health indices and remaining maintenance estimates during a full life-cycle profile.	21
12	AI-agent response during normal operation. The agent summarizes the current health state, RUL estimates, and key sensor readings.	22
13	AI-agent response during a degraded condition with active turbine warning and injected temperature fault.	22

List of Tables

1	Model performance for compressor and turbine decay coefficient prediction.	18
---	--	----

1 Introduction

During the operation of technical systems, components are inevitably subjected to wear and degradation due to continuous use. As this degradation progresses system efficiency and performance gradually decline. If not addressed in a timely manner this deterioration can lead to component failure or complete system breakdown. In marine applications such as the critical propulsion system such failures can result in significant economic losses as well as serious operational and safety consequences.

Gas turbines are widely used in marine vessels as the main propulsion system or as part of combined propulsion configurations. They offer several advantages over conventional marine diesel engines including high power output, compact size, and relatively low weight. These characteristics make gas turbines especially suitable for military vessels and high speed commercial ships where rapid acceleration and high operating speeds are required. The demanding operational environment and continuous loading conditions make gas turbine components particularly susceptible to performance degradation over time.

Predictive maintenance provides an effective means of addressing these challenges by enabling maintenance actions to be scheduled before failures occur. By analyzing operational and condition data, predictive maintenance techniques estimate the future health and remaining useful life of system components allowing maintenance strategies to shift from reactive to proactive. This approach improves system reliability, increases availability, reduces unplanned downtime, and extends the overall service life of propulsion systems.

Condition monitoring plays a central role in predictive maintenance by continuously or periodically collecting sensor data that reflects the operating state of the gas turbine. Key parameters such as shaft torque, rotational speeds, propeller loads, temperatures, pressures, fuel flow, and control settings provide a comprehensive view of the propulsion plant's mechanical and thermal behavior. In addition, structural health monitoring enables the detection of gradual deterioration, such as efficiency losses in the gas turbine compressor or turbine stages, where a drop of 5–10 % in efficiency is typically considered critical. Together these monitoring approaches support safer and more efficient vessel operation, even under challenging environmental or high traffic conditions.

The objective of this work is to implement an AI-Agent using a digital twin model for the predictive maintenance of a gas turbine used as the primary propulsion system of a marine vessel. The study is based on steady state operational data containing gas turbine measurements and decay state coefficients for the compressor and turbine. The sensor data will be preprocessed to handle missing values, remove duplicates, and ensure consistency. Machine learning techniques, including Support Vector Machines, Random Forest, and XGBoost, will then be applied to identify patterns in the data and predict the future condition of the system. Finally, the digital twin will be integrated with an AI-based decision-making agent to support timely and informed predictive maintenance actions.

2 Theory

Predictive maintenance combines condition monitoring with analytical methods to detect degradation, estimate equipment health, and support maintenance before failure occurs. Machine learning is relevant in this context because it can learn relationships directly from data and use them to classify faults, estimate degradation, or predict future conditions. Within digital twin applications, such models can support the interpretation of sensor data and strengthen data-driven decision-making [7, 6].

2.1 Machine Learning Concepts

Machine learning is a branch of artificial intelligence in which algorithms learn patterns from data instead of responding to manually written rules. Rather than specifying every possible input-output relationship in advance, the model is exposed to examples and learns a function that can later be applied to unseen data. The main purpose is to make predictions or decisions from data in a way that generalizes beyond the training set [9, 12]. A useful way to understand machine learning is to distinguish between what the model receives, how it learns, and what its goal is. Based on this, machine learning is commonly grouped into supervised, unsupervised, and reinforcement learning. In addition, semi-supervised and self-supervised learning exist as important extensions.

2.1.1 Supervised Learning

Supervised learning uses labeled data, which means that each training sample contains both an input $\mathbf{x}$ and a desired output y . The model learns from examples where the correct answer is already known [9]. During training, the model learns a mapping from input to output:

$$f(\mathbf{x}) \rightarrow y$$

such that the predicted output $\hat{y}$ is as close as possible to the true target. Once trained, the model can use this learned relationship to make predictions for new data.

2.2 Unsupervised learning

Unsupervised learning uses unlabeled data. In this case, the model only receives the input data $\mathbf{x}$ and no predefined target [12]. The model analyzes the structure of the dataset and searches for regularities on its own. Instead of learning from correct answers, it identifies similarities, differences, or hidden organization in the data. The goal is to discover patterns, structure, or groups that are not explicitly given beforehand.

2.2.1 Reinforcement Learning

Reinforcement learning is a learning paradigm in which an agent interacts with an environment and learns from the consequences of its actions [14]. At each step, the agent observes a state, chooses an action, and receives feedback in the form of a reward or penalty. Over time, it learns a strategy, called a *policy*, that improves long-term performance. The goal is to find a decision strategy that maximizes cumulative reward over time.

2.3 Semi-supervised and self-supervised learning

Semi-supervised learning combines a small amount of labeled data with a larger amount of unlabeled data. This is particularly useful when labeling is expensive or time-consuming.

Self-supervised learning creates supervision from the data itself by defining an auxiliary learning task. The model first learns useful internal representations, which can later be reused in downstream tasks such as classification or regression [9].

2.4 Machine learning models

Based on the problem formulation in this project, the analysis focuses on supervised learning. This is because the datasets used in predictive maintenance typically include known targets, such as fault labels, health states,

or degradation indicators, which allows models to learn the relationship between measured variables and the desired output [11, 6]. Three supervised learning algorithms are used in the analysis: Support Vector Machine, Random Forest, and Extreme Gradient Boosting.

2.4.1 Support vector machine

Support Vector Machine (SVM) is a supervised learning algorithm mainly used for classification, although regression variants also exist. For classification, its purpose is to separate classes by constructing a decision boundary in the feature space [4, 9]. SVM searches for a hyperplane that separates the classes while maximizing the margin between them. For a linear case, the decision boundary can be written as:

$$\mathbf{w}^\top \mathbf{x} + b = 0 \quad (1)$$

where $\mathbf{w}$ is the normal vector and b is the bias. A prediction is then made from:

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b) \quad (2)$$

If the data is not linearly separable, kernel functions can be used to map the data implicitly into a higher-dimensional space where separation becomes easier. The goal of SVM is to create a decision boundary with the best possible separation between classes, which often improves generalization on unseen data. Figure 1 illustrates this principle.

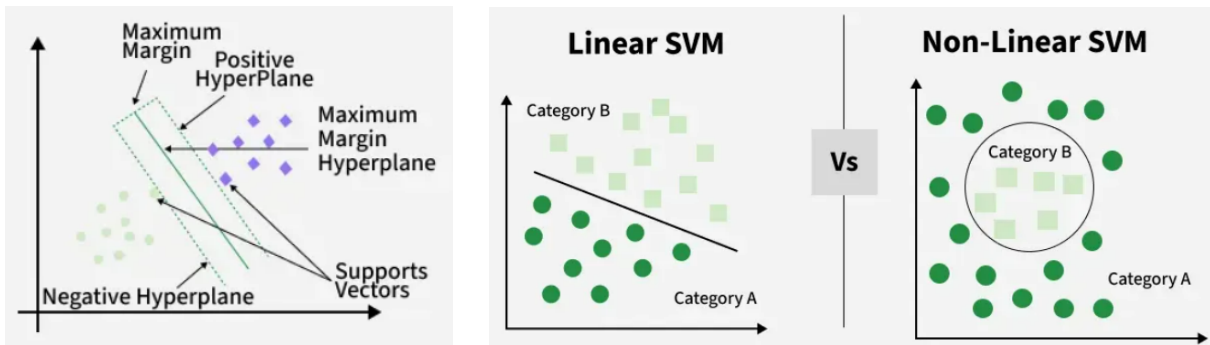


Figure 1: Illustration of the Support Vector Machine principle.

2.4.2 Random forest

Random Forest is a supervised ensemble learning algorithm based on a collection of decision trees, and can be used for both classification and regression [1, 9]. Instead of relying on a single tree, Random Forest builds many trees from different bootstrap samples of the training data. In addition, each split only considers a random subset of the available features. The final prediction is obtained by aggregating the results from all trees, using majority voting for classification and averaging for regression. The goal is to improve predictive performance and reduce overfitting by combining many slightly different decision trees into one stronger model. Figure 2 illustrates the Random Forest principle.

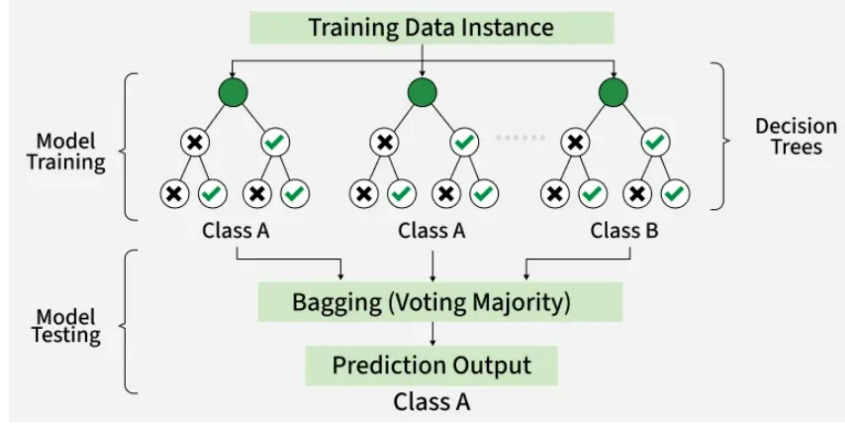


Figure 2: Illustration of the Random Forest principle.

2.4.3 Extreme gradient boosting

Extreme Gradient Boosting (XGBoost) is a supervised ensemble algorithm based on gradient boosting with decision trees, designed for high predictive performance and efficient computation [3]. Unlike Random Forest, which trains trees independently, XGBoost builds trees sequentially. Each new tree is trained to correct the errors made by the current ensemble. The prediction can be expressed as:

$$\hat{y}^{(t)}(\mathbf{x}) = \sum_{k=1}^t h_k(\mathbf{x}) \quad (3)$$

where $h_k(\mathbf{x})$ is the contribution from the k th tree. The goal is to iteratively improve the model by focusing new trees on the residual errors of the previous ones, thereby increasing predictive accuracy. Figure 3 illustrates the XGBoost principle.

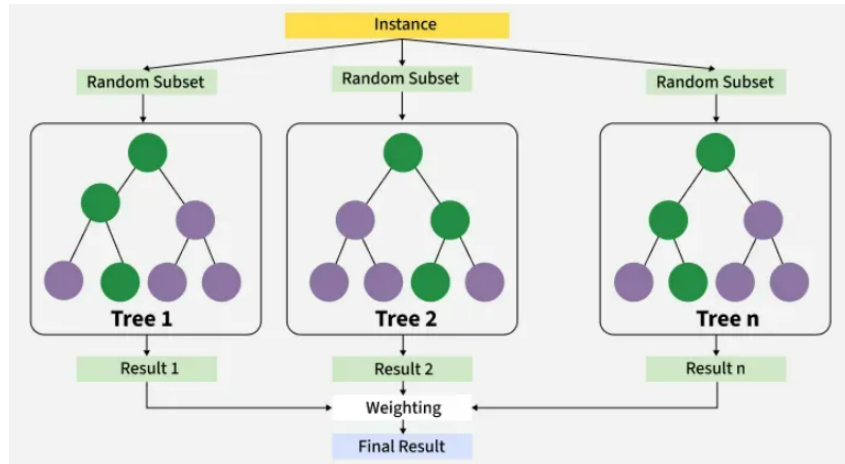


Figure 3: Illustration of the XGBoost principle.

2.5 Large language models

Large Language Models (LLMs) are neural network models trained on very large text corpora to learn patterns, structure, and relationships in human language. In practical terms, an LLM predicts the next token in a sequence based on the previous context. By training on massive amounts of text, the model learns grammar, semantics, style, and task-relevant language patterns without being manually programmed for each individual task. This makes LLMs useful for applications such as summarization, question answering, text generation, information extraction, and conversational systems [2, 15].

The basic principle of an LLM is that it learns a probability distribution over sequences of text. For a sequence of tokens $x_1, x_2, \dots, x_T$, the model learns:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t}) \quad (4)$$

where $x_{<t}$ represents all previous tokens. During training, the model adjusts its internal parameters to improve these predictions. As a result, it develops internal representations that can later be used for many downstream language tasks [2].

In this project, Ollama is used as the local framework for running the language model. Ollama is not a language model itself, but a platform for downloading, managing, and running open large language models locally through a simple interface and API. A local setup provides more control over data handling, allows offline use, and makes it easier to test different open models in a reproducible workflow [13].

2.6 Dashboard framework

A dashboard framework provides the interface layer between analytical models and the user. In predictive maintenance systems, this enables model outputs, sensor-derived information, and language-based interaction to be presented in a structured and accessible form. Rather than changing the underlying predictive model itself, the dashboard acts as the communication layer that allows users to submit inputs, inspect results, and interact with the system in real time.

Gradio is an open-source Python framework designed for building interactive web applications around machine learning models, APIs, and general Python functions [5]. A key characteristic of Gradio is that it allows user interfaces to be defined directly in Python, which makes it well suited for data-driven and model-centered applications where rapid development and clear interaction design are important. Conceptually, Gradio can be understood as a bridge between backend computation and frontend interaction. A Python function is connected to input and output components, and the framework automatically generates a browser-based interface around this functionality.

Gradio provides two main design paradigms. The first is the *Interface* abstraction, which offers a high-level way of wrapping a single function with predefined inputs and outputs. The second is *Blocks*, a lower-level and more flexible system for composing custom dashboards from multiple components. This component-based structure is particularly relevant in engineering applications, since dashboards often need to combine several interface elements such as text boxes, buttons, plots, tables, and chat windows within one unified environment.

2.7 Correlation analysis

Correlation analysis is used to examine the statistical relationship between variables. In machine learning and exploratory data analysis, it is useful for identifying how strongly variables vary together and for providing an initial indication of whether some features may be informative for prediction [9].

A correlation matrix is a tabular representation of pairwise correlation values between variables. When visualized as a heatmap, it provides a compact overview of positive, negative, and weak relationships across the dataset. This makes it easier to detect groups of strongly related variables and possible redundancy among features [10].

2.8 Performance metrics

To evaluate and compare the predictive models, metrics are needed that describe how closely the predicted values match the true target values. Since the present work is based on regression, the evaluation should reflect both the magnitude of the prediction error and how well the model explains variation in the target data

[9, 8].

2.8.1 Mean absolute error

Mean Absolute Error (MAE) measures the average absolute difference between the predicted values and the true target values, and is calculated as:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

where y_i is the true value, $\hat{y}_i$ is the predicted value, and N is the number of observations. MAE provides a direct measure of the average prediction error in the same unit as the target variable.

2.8.2 Coefficient of determination

The coefficient of determination, denoted R^2 , measures how much of the variation in the target variable is explained by the model. It is defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

where y_i is the true value, $\hat{y}_i$ is the predicted value, and $\bar{y}$ is the mean of the true target values. R^2 quantifies how well the model captures the variability in the observed data, with values closer to 1 indicating stronger explanatory performance [8].

3 Methodology

This chapter describes the methodology used to develop and evaluate the predictive maintenance models for the marine vessel propulsion dataset. The workflow was divided into four main stages: data description, data preparation, model training, and model evaluation. The purpose was to establish a consistent and reproducible workflow from raw operational data to evaluated machine learning models, which could later be used in the digital twin for health monitoring and predictive maintenance.

3.1 Data description

The dataset describes a marine vessel propulsion plant based on a gas turbine. It consists of steady-state operational measurements from the propulsion system. The input variables include lever position, ship speed, gas turbine shaft torque, gas turbine revolutions, gas generator revolutions, starboard and port propeller torque, turbine temperature, pressure measurements, turbine injection control, and fuel flow.

These variables describe both the operating condition of the propulsion system and the mechanical and thermodynamic response of the gas turbine. For example, lever position and ship speed describe the commanded operating state, while torque, revolutions, temperature, pressure, and fuel flow describe how the propulsion system responds under load.

The two target variables are the gas turbine compressor decay state coefficient and the gas turbine decay state coefficient. These coefficients represent the health condition of the compressor and turbine. A coefficient close to 1 indicates a healthy component, while a lower value indicates degradation. According to the assignment description, a drop of approximately 5–10% in efficiency may be considered critical for gas turbines.

In this project, the machine learning task was treated as a supervised regression problem. The operational sensor variables were used as input features, while the compressor and turbine decay state coefficients were used as target values.

3.2 Data preparation

Before model training, the dataset was prepared to make it suitable for machine learning. The original dataset ordering showed the compressor and turbine decay coefficients increasing through the dataset. Since the purpose of this project was to model degradation for predictive maintenance, the order of the dataset was reversed so that the coefficients followed a degradation direction from a healthy state towards a more degraded state. This was not done to change the relationship between the variables, but to make the chronological interpretation of the degradation trend more suitable for the digital twin and maintenance logic.

The column names were then cleaned by removing brackets, spaces, and special symbols. This made the variable names easier to handle in Python and reduced the risk of errors during preprocessing and model training. The dataset was also checked for missing values, duplicated rows, and constant columns.

The following target columns were removed from the input feature set and stored separately as target variables:

- GT Compressor decay state coefficient
- GT Turbine decay state coefficient

This was done to prevent data leakage. If the target columns had remained in the input feature set, the models could have learned directly from the answer instead of learning the relationship between the sensor variables and component degradation.

Constant or non-informative columns were also removed when identified during data inspection. These columns do not provide useful information to the models because their values do not change across the dataset. In addition, highly correlated variables were investigated using a correlation matrix to better understand dependencies between features. The correlation matrix was used as a supporting tool for data understanding, not as the only basis for feature selection.

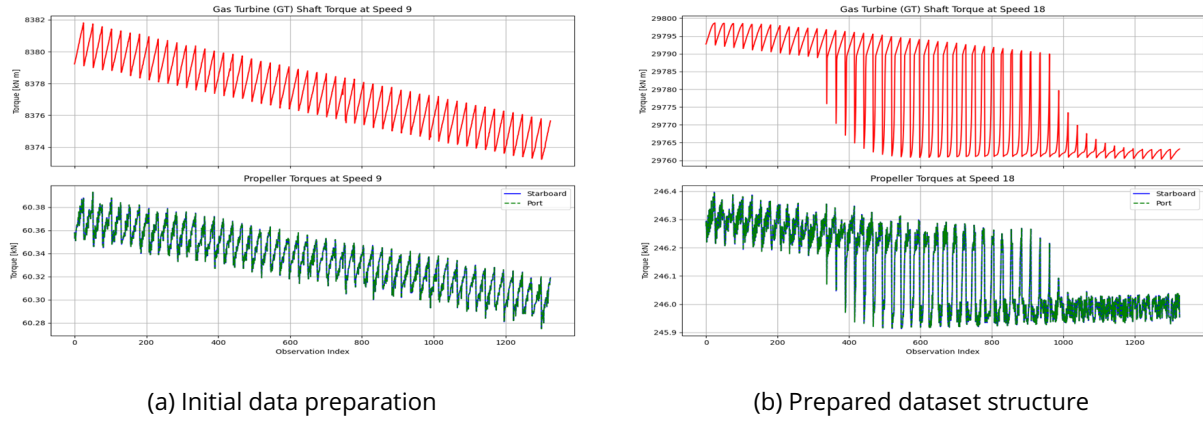


Figure 4: Data preparation workflow before model training.

To inspect consistency in the dataset, selected variables such as torque, propeller torque, and fuel flow were plotted at fixed operating states. This made it possible to check whether repeated operating conditions followed similar patterns and to identify possible irregular behaviour before model training.

The dataset was then split into training and test data using an 80/20 split. This split was chosen because it gives the models enough data to learn the relationship between the sensor variables and the decay coefficients, while still keeping a separate test set for evaluating performance on unseen data. A fixed random state was used to make the split reproducible.

Feature scaling was applied for the SVM model because SVM is sensitive to differences in variable magnitude. Without scaling, variables with large numerical values, such as torque or revolutions, could dominate variables with smaller numerical values, such as pressure or fuel flow. Tree-based models such as Random Forest and XGBoost were not scaled, because they split the data based on feature thresholds and are less sensitive to feature magnitude.

3.3 Model training and configuration

Three supervised regression models were trained and compared in this project: Random Forest, Support Vector Machine (SVM), and XGBoost. These models were selected because they can model nonlinear relationships between propulsion system variables and component health. The same prepared training and test datasets were used for all models to make the comparison as fair as possible.

The Random Forest model was configured with 100 trees and a fixed random state. Using multiple trees makes the model more robust by averaging predictions from many decision trees, while the fixed random state ensures reproducible results.

The SVM model was trained using a radial basis function (RBF) kernel. This kernel was selected because it allows the model to capture nonlinear relationships in the dataset. Since SVM is sensitive to feature magnitude, the scaled version of the dataset was used for this model.

The XGBoost model was configured with 100 estimators and a learning rate of 0.1. XGBoost builds decision trees sequentially, where each new tree attempts to correct errors from the previous trees. This makes the method suitable for regression problems where small prediction errors are important.

The hyperparameter choices were kept relatively simple and were mainly used as baseline configurations for comparing the three model types. A full hyperparameter optimization was not performed in this project. This means that the comparison focuses on how the selected model types performed with reasonable initial settings, rather than finding the absolute optimal configuration for each model.

For each algorithm, separate models were trained for the compressor and turbine decay state coefficients. This was done because the compressor and turbine are different components and may have different degradation behaviour. The trained models were then saved and compared using the evaluation approach described in the next section.

3.4 Model evaluation approach

The trained models were evaluated by comparing their predicted decay coefficients against the true compressor and turbine decay state coefficients in the test dataset. Each model was evaluated separately for both target variables. The test dataset was not used during training, which allowed the evaluation to measure how well the models generalized to unseen data.

The main evaluation metrics were mean absolute error (MAE) and the coefficient of determination (R^2). MAE was used to measure the average size of the prediction error, where a lower value indicates better prediction accuracy. R^2 was used to measure how much of the variation in the target variable was explained by the model, where a value closer to 1 indicates a better fit.

In addition to the numerical metrics, predicted values were plotted against the actual values. These plots were used to visually inspect how well each model followed the degradation trend. The final model selection was based on both the numerical metrics and the visual agreement between predicted and actual decay behaviour.

4 Digital Twin Implementation

A digital twin is a virtual representation of a physical system that evaluates the system state using available operational data. In this project, the digital twin combines trained machine learning models with monitoring logic to track component health, detect maintenance events, estimate remaining useful life, and provide possible diagnostic indications. The digital twin is also connected to an AI-agent and a Gradio dashboard to make the results easier to interpret for the user.

4.1 Digital Twin System Architecture

The digital twin was implemented as a Python class that receives sensor data as input and produces health assessments as output. The system follows the pipeline shown in Figure 5. Sensor data is first preprocessed, then passed through the selected machine learning models. The predicted compressor and turbine decay coefficients are then used by the digital twin modules for health monitoring, maintenance event detection, possible root cause indication, and remaining useful life estimation.

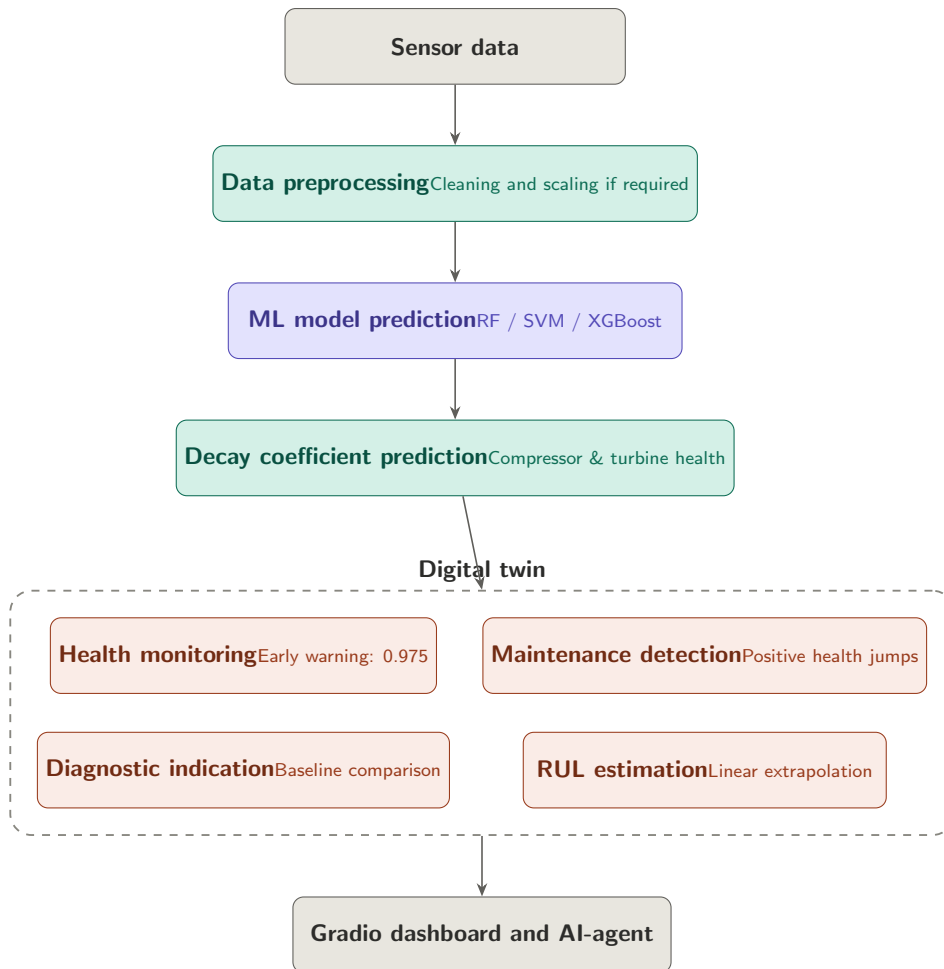


Figure 5: Digital twin system architecture. Sensor data flows through preprocessing and machine learning prediction into the digital twin, which performs health monitoring, maintenance detection, diagnostic indication, and RUL estimation before presenting the results in the Gradio dashboard and AI-agent interface.

The digital twin is initialized with the best performing compressor and turbine models selected during the model evaluation stage. If the selected model requires feature scaling, such as the SVM model, the corresponding scaler object is also loaded. This makes it possible to use different trained models without changing the monitoring logic.

The digital twin maintains an internal state consisting of health history, a maintenance event log, and cycle tracking variables. Each time new sensor data is received, the digital twin predicts the compressor and turbine

decay coefficients, stores the values in the health history, and checks for threshold violations, maintenance events, and degradation trends.

4.2 Model Prediction

The model prediction module is responsible for converting incoming sensor data into estimated component health values. The input to this module is one row of operational sensor data containing the same features that were used during model training. Before prediction, the input data is arranged in the correct feature order to match the training dataset.

If the selected model requires scaled input, the stored scaler is applied before prediction. This is especially important for the SVM model, since SVM is sensitive to differences in feature magnitude. For Random Forest and XGBoost, the raw feature values are used directly because tree-based models are less sensitive to scaling.

Two separate predictions are made for each input sample:

- Compressor decay state coefficient
- Turbine decay state coefficient

The output values are interpreted as health indicators. A value close to 1.0 represents a healthy component, while lower values indicate degradation. These predicted values are then passed to the health monitoring, maintenance detection, diagnostic indication, and RUL modules.

4.3 Health Monitoring

The primary function of the digital twin is to monitor the health of the compressor and turbine by comparing the predicted decay state coefficients against a predefined warning threshold. In this project, the early warning threshold is set to 0.975, corresponding to approximately 2.5% degradation from the nominal healthy state of 1.0.

The assignment description states that a drop of approximately 5–10% in gas turbine efficiency may be considered critical. This means that values around 0.95 or lower indicate a more severe degradation level. The threshold of 0.975 was therefore selected as a conservative early warning limit, not as the final critical failure limit. The purpose is to notify the user before the system reaches a more critical degradation level.

For each prediction, the digital twin evaluates two main conditions:

- If the compressor decay coefficient falls below 0.975, a compressor warning is generated.
- If the turbine decay coefficient falls below 0.975, a turbine warning is generated.

The health monitoring module also tracks the trend direction for each component. A linear regression is fitted to the historical decay coefficients to estimate the slope. A negative slope indicates degradation, a positive slope indicates recovery, and a near-zero slope indicates stable operation. This trend information is displayed together with the current health value in the dashboard.

4.4 Maintenance Event Detection

Gas turbine maintenance can restore component performance partially or fully. This may create a sawtooth-like pattern in the health data, where the decay coefficient gradually decreases during operation and then increases after maintenance. The digital twin detects possible maintenance events by identifying sudden positive jumps in the predicted health values.

The detection mechanism compares the current predicted decay coefficient with the previous value. If the current value is higher than the previous value by more than a predefined jump threshold of 0.01, the system interprets this as a possible maintenance event. When such an event is detected, the digital twin records:

- The sample index where the event occurred.
- The duration of the preceding degradation cycle, measured in number of samples.
- The recovery level after the event.

- Whether the recovery appears to be full or partial.

A recovery is classified as *full* if the health coefficient returns above 0.99, and *partial* if it remains below this value. This classification is a simplified interpretation and should be treated as a support indicator rather than a confirmed maintenance assessment.

4.5 Possible Root Cause Indication

The digital twin includes a simple diagnostic module that provides possible root cause indications. This module does not perform confirmed fault diagnosis. Instead, it highlights which sensor values deviate most from a healthy baseline.

The baseline is calculated from data points where the compressor and turbine decay coefficients are close to healthy operation. In this implementation, healthy operation is represented by samples where the decay coefficient is above 0.99. The most recent sensor readings are then compared against this baseline.

The selected diagnostic variables are:

- HP turbine exit temperature T_{48}
- Compressor outlet temperature T_2
- HP turbine exit pressure P_{48}
- Compressor outlet pressure P_2
- Fuel flow $\dot{m}_f$

For each sensor, the percentage deviation from the healthy baseline is calculated as:

$$\text{Deviation} = \frac{x_{\text{current}} - x_{\text{baseline}}}{x_{\text{baseline}}} \times 100\% \quad (5)$$

The sensors with the largest absolute deviations are highlighted as possible contributors to the observed degradation. For example, abnormal temperature, pressure, or fuel flow values may indicate that the gas turbine is operating differently from the healthy baseline. However, this method only provides an indication of possible causes. It cannot confirm the actual physical root cause without additional analysis, domain knowledge, or inspection data.

4.6 Remaining Useful Life Estimation

To support maintenance planning, the digital twin estimates the remaining useful life (RUL) of the compressor and turbine. The RUL estimation is based on a linear extrapolation of the observed degradation trend. A first-degree polynomial is fitted to the historical decay coefficients using least-squares regression. The slope from this regression represents the average degradation rate per sample.

The remaining useful life is estimated as the number of samples until the predicted health value crosses the early warning threshold:

$$\text{RUL} = \frac{\theta_{\text{threshold}} - \theta_{\text{current}}}{s} \quad (6)$$

where θ_{current} is the current decay coefficient, $\theta_{\text{threshold}}$ is the warning threshold of 0.975, and s is the estimated degradation slope per sample.

This method is a simplified RUL estimate. It assumes that the degradation trend continues linearly, which may not be true for real gas turbine degradation. In practice, degradation can be nonlinear and affected by operating conditions, maintenance actions, load changes, and measurement uncertainty. Therefore, the RUL value should be interpreted as a trend-based estimate rather than an exact prediction.

4.7 Dashboard Implementation

As introduced in Section ??, the dashboard was developed using the open-source Python framework Gradio. The purpose of the dashboard is to present the digital twin outputs in a clear and interactive format.

The dashboard includes:

- Current compressor and turbine health values.
- Warning indicators based on the 0.975 early warning threshold.
- Historical health trend plots.
- Maintenance event log.
- RUL estimates.
- Sensor deviation plots for possible diagnostic indication.
- AI-agent chat interface.

The dashboard allows the user to inspect both current and historical system behaviour. Time-series plots are used to show how the compressor and turbine health values change over time. The maintenance log records detected positive health jumps, while the diagnostic view highlights which sensor values deviate most from healthy baseline operation.

4.8 AI-Agent Purpose and Design

The AI-agent was implemented as a decision-support tool connected to the digital twin. Its purpose is to help the user interpret the digital twin outputs in natural language. Instead of only showing numerical values, the AI-agent can summarize the current system state, explain warnings, describe possible degradation trends, and suggest maintenance-related actions.

The AI-agent was implemented using a locally hosted large language model through Ollama. A local model was selected to demonstrate how an AI-assistant can be integrated without depending on an external cloud service. The agent does not train the machine learning models and does not directly change the digital twin state. It only receives structured information from the digital twin and converts it into readable explanations.

4.9 AI-Agent Inputs and Outputs

The AI-agent receives a structured context from the digital twin. The input context includes:

- Current compressor decay coefficient.
- Current turbine decay coefficient.
- Current warning status.
- Recent health trend direction.
- Estimated RUL.
- Latest maintenance events.
- Largest sensor deviations from baseline.
- User question from the dashboard chat window.

Based on this information, the AI-agent produces a text response. The output is intended to be understandable for an operator or engineer and may include:

- A short summary of the current component health.
- Explanation of warning messages.
- Interpretation of degradation or recovery trends.

-
- Possible sensor-related causes of abnormal behaviour.
 - Maintenance recommendations or inspection suggestions.

The AI-agent output is advisory and should not be interpreted as a verified fault diagnosis. The response depends on the digital twin outputs and the quality of the underlying sensor data and machine learning predictions.

4.10 Prompt Structure

The AI-agent uses a system prompt that defines its role as a maintenance support assistant for a marine gas turbine propulsion system. The prompt instructs the model to base its answers only on the provided digital twin context and to avoid presenting uncertain indications as confirmed faults.

A simplified version of the prompt structure is:

You are an engineering assistant for a marine vessel propulsion digital twin. Use the provided compressor health, turbine health, RUL estimate, maintenance log, and sensor deviations to answer the user's question. Explain warnings clearly, but do not claim confirmed root causes unless the data supports it. Treat sensor deviations as possible indications only.

The user question is then added together with the current digital twin state. This makes the AI-agent response specific to the current operating condition instead of being a general text response.

4.11 Example AI-Agent Response

An example of a user question could be:

Why is the turbine warning active?

A typical AI-agent response may be:

The turbine warning is active because the predicted turbine decay coefficient is below the early warning threshold of 0.975. This does not mean that the turbine has failed, but it indicates that the predicted health has moved into the warning region. The current degradation trend should be monitored, and the sensor deviation view should be checked for abnormal temperature, pressure, or fuel flow behaviour. Since the diagnostic module is based on baseline comparison, the highlighted sensors should be treated as possible indications rather than confirmed root causes.

This demonstrates how the AI-agent combines model prediction, threshold monitoring, RUL, and diagnostic indications into a more readable explanation for the user.

5 AI-Agent Prediction

5.1 AI-Agent Purpose and design rationale

The primary reason for AI-Agent implementation is interpretability. The digital twin outputs numerical decay coefficients, gradient slopes, sensor deviations, and maintenance logs. While all of these values are precise, someone monitoring the system must combine multiple data streams to form a coherent picture of the system's condition. For example, understanding that a compressor decay coefficient of 0.971 combined with a rising turbine exit temperature and increasing fuel flow likely indicates compressor fouling requires cross-reference several parameters simultaneously.

the AI agent addresses this by synthesizing the available data into structured natural language response. Rather than replacing the numerical outputs, the agent acts as a decision support layer that translates data into insights. The user can query the system with free-text questions and receive responses that combine current sensor readings, degradation trends, maintenance history, and diagnostics information into a single coherent analysis.

5.2 AI System Architecture

The AI-Agent system follows a pipeline consisting of four stages: context assembly, prompt construction, language model inference, and response delivery.

When a user submits a query through the dashboards's chat interface, the system first assembles a snapshot of the current digital twin state. This snapshot is combined with the user's question into a structured prompt. The prompt is sent to a locally hosted Large Language Model, or LLM, through the Ollama framework. The model generates a response, which is streamed back to the chat interface. A dashed feedback loop in the architecture allows for follow-up questions, with conversation history saved.

5.3 Context assembly

The quality of the AI Agent's response depends directly on the quality of the context it receives. Each time the user submits a query, a context snapshot is computed from the currently filtered dashboard data. This snapshot is assembled by the `compute_chat_snapshot` function and contains the following information:

- **Operating conditions:** The latest values of ship speed, GT shaft torque, fuel flow, turbine injection control, and rotational speeds.
- **Temperature and pressure readings:** Current values of T_{48} , T_2 , P_2 , P_{48} , and other sensor measurements.
- **Health status:** The mean compressor and turbine decay coefficients for the filtered window, classified as HEALTHY (above 0.975) or CRITICAL (below 0.975).
- **Degradation trends:** The linear slope of each decay coefficient expressed per sample, and the estimated remaining useful life derived from this slope (see section ??).
- **Maintenance history:** A summary of the most recent maintenance events detected by the digital twin, including cycle duration, recovery level, and whether the recovery was full or partial.
- **Sensor diagnostics:** The top sensor deviations from the healthy baseline, expressed as percentage changes. This information is produced by the digital twin's root cause diagnostics module (see section ??).

The context assembly includes a caching mechanism to avoid redundant computation. A fingerprint is computed from the filtered dataframe based on its length, column names, and a sample key values. If the fingerprint matches a previously computed snapshot, the cached result is reused. This reduces latency when the user asks multiple questions without changing the dashboard filters.

5.4 Agent capabilities

The AI Agent acts as an interactive Chief Engineer, designed to bridge the gap between raw numerical outputs and human readable maintenance insights. It is equipped with several core capabilities that synthesize com-

plex telemetry data into actionable advice. First, it performs **health status interpretation**. If the digital twin outputs a coefficient of 0.982, the agent translates this into a contextual insight, explaining to the operator that the compressor is at 98.2% health and is operating within the normal range. Second, it conducts **degradation trend analysis**. By evaluating the linear regression on historical decay coefficients, the agent can communicate the estimated RUL in plain language, indicating exactly how many mission cycles remain before a critical threshold is crossed.

Third, the agent provides **maintenance history evaluation**. It can assess past maintenance events to determine if a recent intervention resulted in a full or partial recovery, and explain what a partial recovery implies for future degradation rates. Fourth, it offers **root cause suggestions** by linking specific sensor deviations to potential failure modes. For instance, the agent can identify that an unexpected spike in turbine exit temperature (T_{48}) combined with an increased fuel flow points toward likely compressor fouling. Finally, it delivers concrete **maintenance recommendations**, advising operators on exactly what physical inspections or cleaning actions need to be taken, and when they should be scheduled to avoid unscheduled downtime.

5.5 Prompt engineering

To ensure the language model provides accurate and highly contextual responses, the system utilizes dynamic system prompting. Each time a user submits a query, the application generates a real time system prompt that defines the agent's persona as an AI Chief Engineer and strictly injects the current system state. The instructions the LLM receives include the current ship speed, the active mission cycle, the calculated health indices, and the RUL for both the compressor and the gas turbine. Furthermore, it is supplied with a snapshot of precise sensor readings, including torque, RPM, fuel flow, turbine exit temperature, and compressor outlet pressure, alongside a detailed string of any active, user injected faults.

To ensure the model stays factual and does not hallucinate, the prompt enforces strict constraints on the response format. The LLM is explicitly directed to avoid generic AI disclaimers, maintain a direct and analytical tone, and strictly reference only the numerical sensor values provided within the prompt when answering technical questions. By forcing the LLM to ground its responses exclusively in the dynamically injected telemetry data, the risk of the model inventing operational parameters is mitigated.

5.6 Integration with dashboard

The AI Agent is seamlessly integrated into the Gradio dashboard as a dedicated tab using the framework's `ChatInterface` abstraction. Users interact with the agent by typing free text questions natively within this window, much like a standard messaging application. Because the chat interface is housed within the unified dashboard, operators can consult the AI while simultaneously navigating to adjacent tabs to view the digital twin's physical gauges, historical trend plots, and sensor grids.

To maintain state consistency between the digital twin and the language model, the system utilizes Gradio's state management capabilities. Whenever a user selects a new mission cycle or manually injects a fault, the dashboard's update function recalculates the current snapshot, health metrics, and RUL. These variables are passed automatically as hidden state inputs to the function connecting to the AI agent, ensuring the LLM is always analysing the exact data the user is currently viewing. The inference layer runs locally using the Ollama framework, specifically utilizing models such as Llama 3. This local setup ensures that sensitive telemetry data does not leave the vessel's internal network. Communication between the dashboard and Ollama is handled via a streaming API, allowing the agent's responses to be streamed token-by-token back to the chat interface, which provides a highly responsive and real time user experience.

6 Results and Discussion

This chapter presents the results from the data exploration, model comparison, digital twin demonstration, and AI-agent demonstration. The purpose is to show how the prepared dataset was analysed, how the trained models performed, and how the final system supports predictive maintenance decisions.

6.1 Data exploration results

The data exploration was used to inspect the structure of the propulsion dataset before model comparison. Selected variables were plotted at fixed speed states to check for repeated operating patterns, as shown in Figure 6.

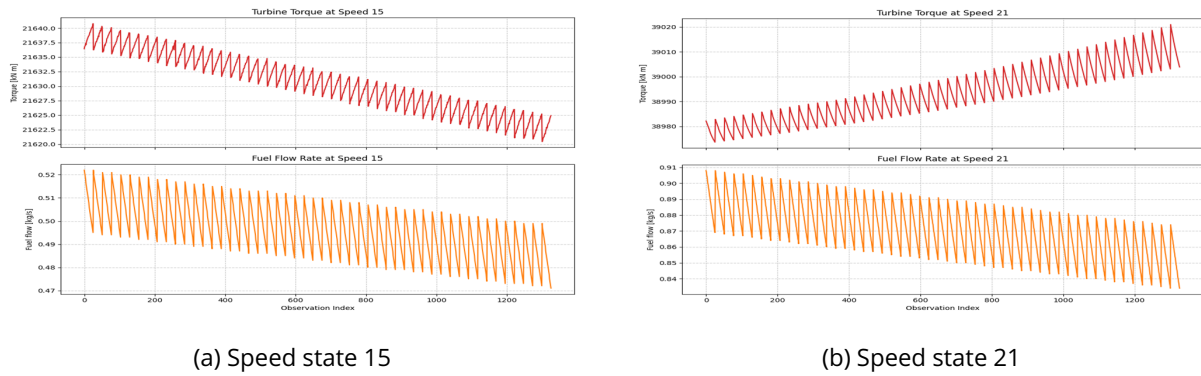


Figure 6: Turbine torque and fuel flow rate at selected speed states.

At speed state 15, both turbine torque and fuel flow show repeated cyclic behaviour with a gradual decreasing trend. At speed state 21, the fuel flow also decreases, while the turbine torque increases over the observation index. This shows that the dataset contains structured operating patterns, but also that the relationship between variables changes between different speed states. Because of this, the full operating condition must be considered rather than evaluating one variable alone.

A correlation matrix was then used to examine linear relationships between the propulsion variables. The purpose was to identify highly related variables and to better understand the structure of the input data before comparing the machine learning models.

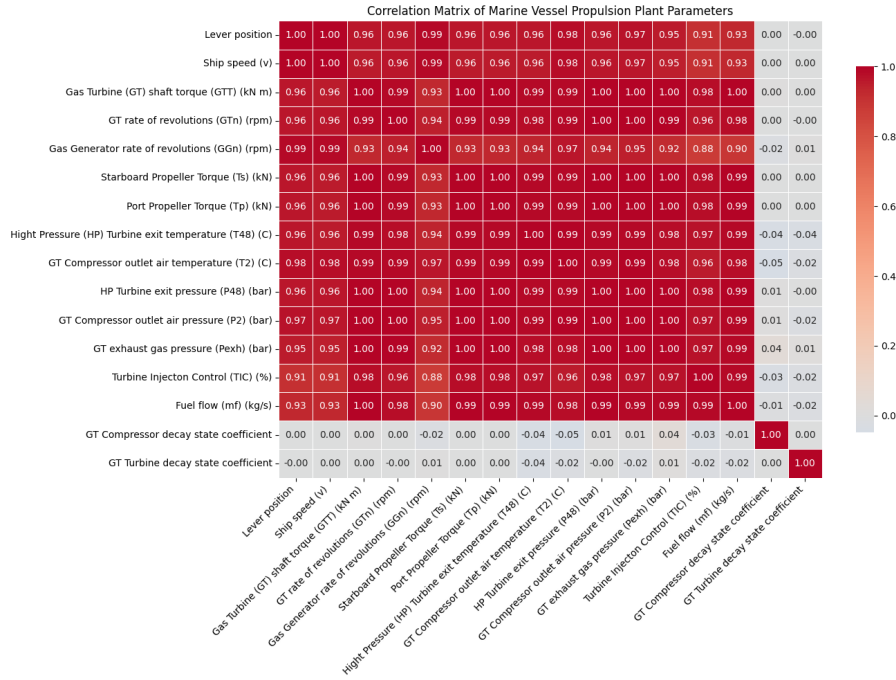


Figure 7: Correlation matrix for the propulsion dataset.

Figure 7 shows that many of the operational variables are strongly correlated. This is expected because variables such as ship speed, torque, revolutions, pressure, temperature, and fuel flow are physically connected through the gas turbine operating state. The compressor and turbine decay state coefficients show lower direct linear correlation with most individual input variables. This indicates that the degradation state is not explained by one sensor variable alone, but by the combined relationship between several operating parameters.

This observation supports the use of machine learning models, since the models can learn nonlinear and combined relationships between several features. The correlation matrix was therefore useful for understanding the dataset, but it was not used alone to decide which variables should be included in the final models.

6.2 Model comparison

The project trained three machine learning models to predict the health coefficients for both compressor and turbine degradation. The models were evaluated against the real decay coefficient values from the provided dataset. Each model was evaluated using the coefficient of determination (R^2) and mean absolute error (MAE). The script also saved the best performing model separately for compressor and turbine prediction, so the selected models could be reused in the digital twin without retraining.

Table 1: Model performance for compressor and turbine decay coefficient prediction.

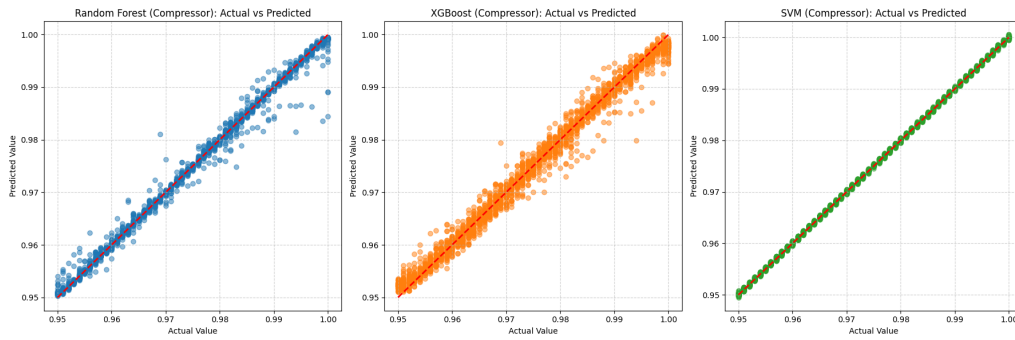
Component	Model	Train R^2	Test R^2	Train MAE	Test MAE
Compressor	Random Forest	0.999364	0.993813	0.000185	0.000525
	XGBoost	0.992804	0.987743	0.000922	0.001131
	SVM	0.999693	0.999695	0.000215	0.000212
Turbine	Random Forest	0.998789	0.989206	0.000127	0.000358
	XGBoost	0.983183	0.969425	0.000666	0.000824
	SVM	0.998434	0.998498	0.000259	0.000253

As shown in Table 1, the best performing model on the test dataset was SVM. For the compressor, SVM achieved a test MAE of 0.000212 and a test R^2 of 0.999695. For the turbine, SVM achieved a test MAE of 0.000253 and a test R^2 of 0.998498. This indicates that the SVM model was able to predict both decay coefficients with very low error on the test data.

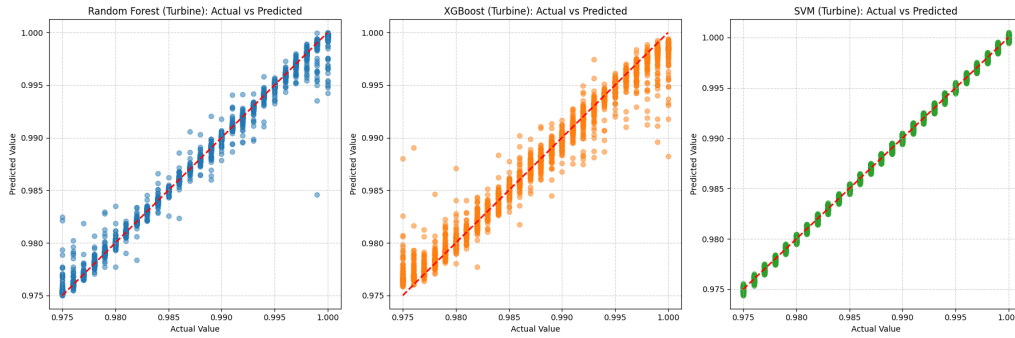
The difference between training and testing performance was also small for SVM. For the compressor, the test MAE was slightly lower than the training MAE. This should not be interpreted as the model becoming better during testing, but rather as a result of the random train-test split where the test subset may have been slightly easier to predict. The important observation is that the training and testing scores are close, which indicates good generalization within the available dataset.

Random Forest also performed well, especially during training. However, the increase from train MAE to test MAE was larger than for SVM. This indicates that Random Forest may have fitted the training data more closely, while SVM gave more stable performance on unseen test samples. XGBoost gave the weakest performance of the three models for both compressor and turbine prediction in this setup.

The very high R^2 values should also be interpreted critically. One likely reason for the high scores is that the dataset is structured and based on steady-state operating conditions with repeated operational patterns. In addition, random splitting can place very similar operating samples in both the training and test sets. This can make the test task easier than a fully chronological or mission-based split. Therefore, the model performance is strong for this dataset, but the results should not automatically be interpreted as proof that the same accuracy would be achieved on completely new vessel data.



(a) Compressor predictions compared against actual values.



(b) Turbine predictions compared against actual values.

Figure 8: Prediction performance for compressor and turbine decay coefficients.

Figure 8 shows the predicted values compared with the actual decay coefficients. The SVM predictions follow the actual values closely for both compressor and turbine. Random Forest also follows the general trend, but with larger deviations than SVM. XGBoost shows the largest deviations in this comparison.

The compressor prediction appears more accurate than the turbine prediction for all models. This may be related to differences in the degradation pattern. The compressor degradation follows a more continuous trend, while the turbine data contains more repeated degradation and recovery behaviour. This makes the turbine prediction more challenging, especially if maintenance-like changes create sudden shifts in the decay coefficient.

Based on the numerical metrics and the predicted versus actual plots, SVM was selected as the final prediction model for both compressor and turbine health in the digital twin.

6.3 Digital twin demonstration

The digital twin demonstration shows how the selected machine learning models were used as part of a predictive maintenance system. The dashboard converts the model predictions into health values, warning states, maintenance estimates, sensor information, and visual trends. This makes the machine learning output easier to interpret as an operational monitoring tool.

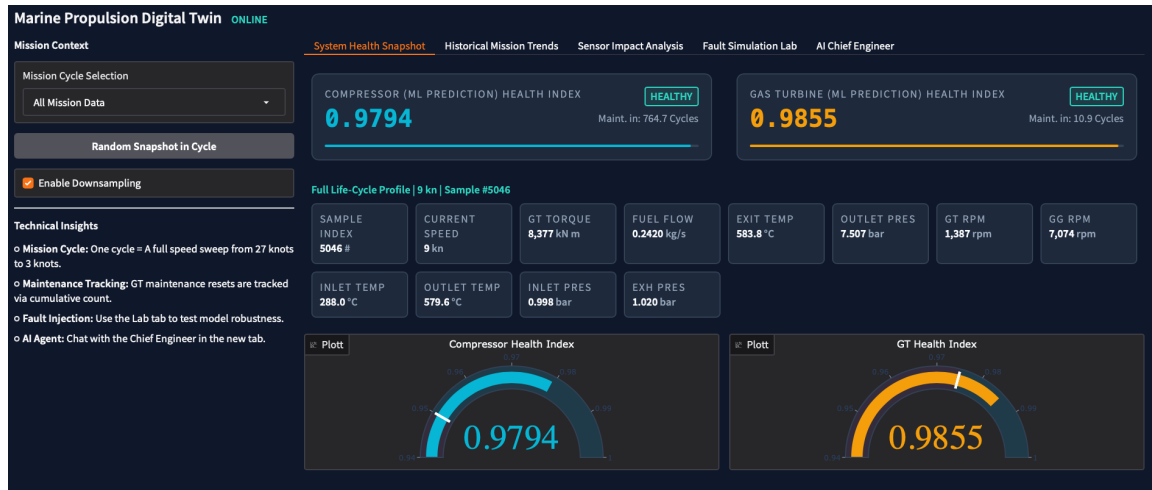


Figure 9: Digital twin dashboard during normal operation. Both compressor and gas turbine are above the warning threshold and marked as healthy.

Figure 9 shows a healthy operating snapshot from the dashboard. The compressor health index is predicted as 0.9794, while the gas turbine health index is predicted as 0.9855. Both values are above the early warning threshold of 0.975 and are therefore marked as healthy. The dashboard also displays the current operating point, including speed, torque, fuel flow, temperatures, pressures, and rotational speeds. This allows the user to connect the predicted component health to the current operating condition.

The dashboard also estimates how many cycles remain before maintenance is expected. In the shown example, the compressor has an estimated maintenance interval of 164.7 cycles, while the gas turbine has an estimated value of 10.9 cycles. These values are based on the internal health trend and should be treated as trend-based support values rather than exact lifetime predictions.

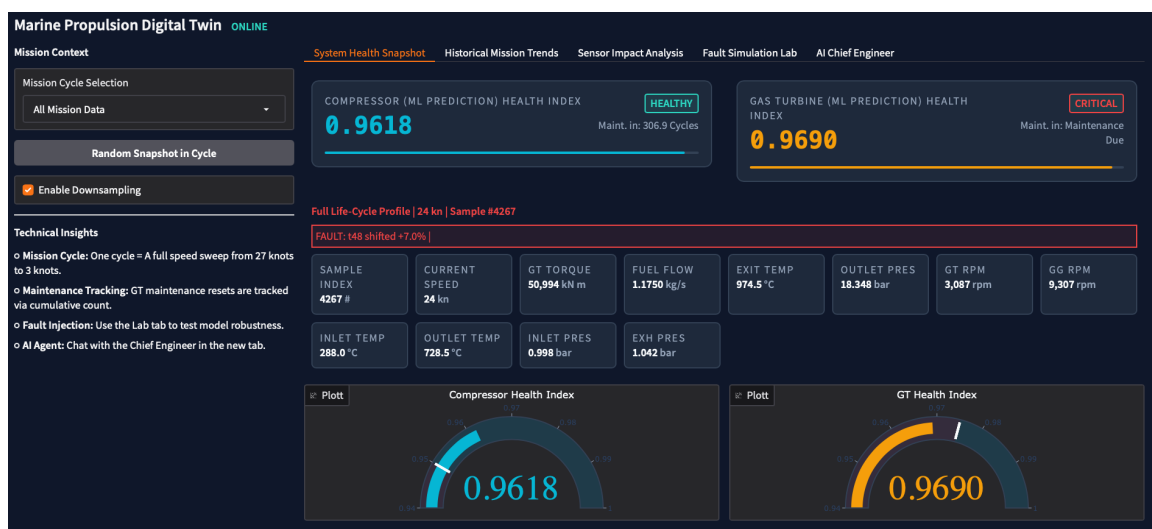


Figure 10: Digital twin dashboard with an injected turbine fault. The gas turbine health index falls below the warning threshold and is marked as critical.

Figure 10 shows the dashboard after a simulated fault injection. In this example, the turbine exit temperature T_{48} has been shifted by 7%, which is shown in the red fault message. The predicted gas turbine health index drops to 0.9690 and is marked as critical because it is below the 0.975 warning threshold. The compressor health index remains higher at 0.9618 and is still marked as healthy in this dashboard state.

This demonstrates that the digital twin reacts to changes in the sensor input and updates the predicted health state. The dashboard does not only show the numerical health index, but also gives an operational warning when the turbine enters the maintenance region. This is important because it turns the machine learning prediction into a practical decision-support signal.

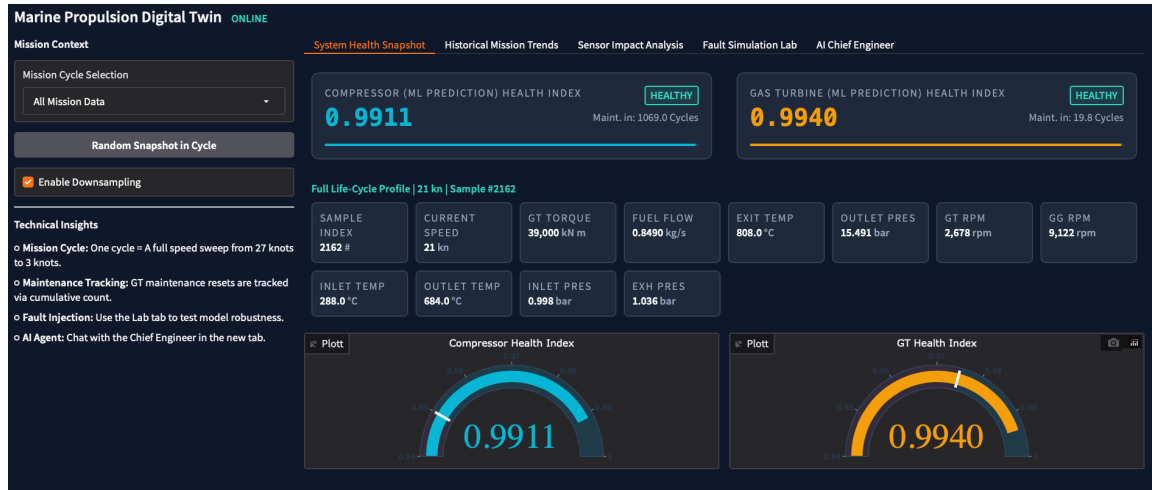


Figure 11: Digital twin dashboard at another mission point, showing health indices and remaining maintenance estimates during a full life-cycle profile.

Figure 11 shows another point in the mission profile. In this case, the compressor health index is 0.9911 and the gas turbine health index is 0.9940. Both components are marked as healthy. The view also shows that the dashboard can be used across a full life-cycle profile and not only for a single isolated operating point.

Overall, the digital twin demonstration shows four main functions. First, it receives operational sensor data from the propulsion dataset. Second, it uses the trained SVM models to predict compressor and turbine health. Third, it compares the predicted health values against the selected warning threshold. Finally, it presents the results through a dashboard with health cards, gauge plots, maintenance estimates, and sensor readings.

The early warning threshold of 0.975 should be interpreted as a conservative maintenance warning level. It does not represent complete failure. Since a 5–10% efficiency drop is considered critical for gas turbines, the 2.5% warning threshold gives the system time to alert the user before reaching a more severe degradation state.

6.4 AI-agent demonstration

The AI-agent demonstration shows how the digital twin information can be presented through a natural language interface. The purpose of the AI-agent is to make the dashboard easier to interpret by summarizing the current system state, warning status, RUL estimate, and sensor behaviour.

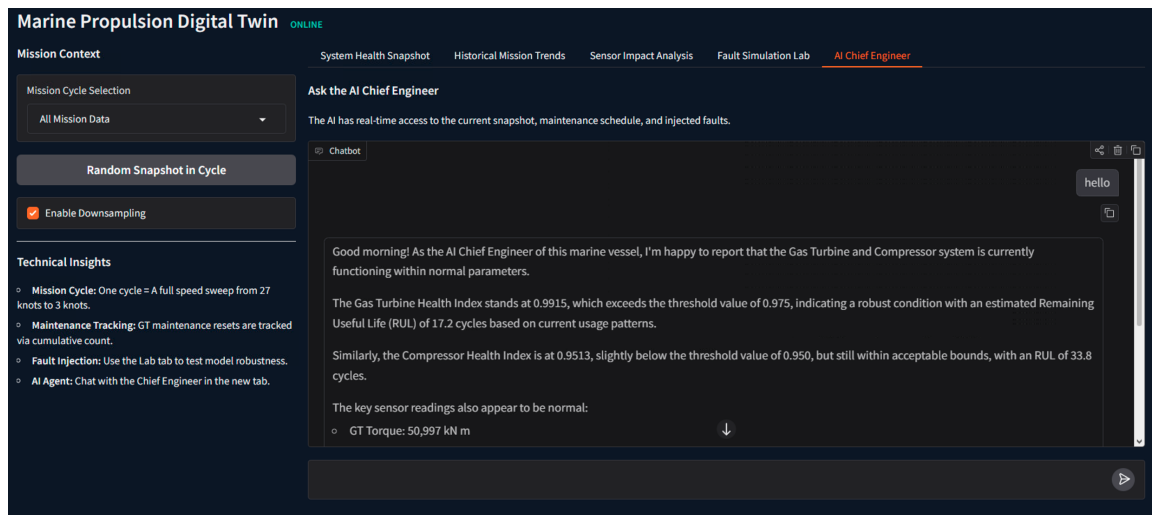


Figure 12: AI-agent response during normal operation. The agent summarizes the current health state, RUL estimates, and key sensor readings.

Figure 12 shows an example where the user asks the AI-agent for a system update. The agent uses the current digital twin context and reports that the gas turbine and compressor are operating within normal parameters. The response includes the gas turbine health index, compressor health index, estimated RUL values, and key sensor readings such as torque, RPM, fuel flow, turbine exit temperature, and compressor outlet pressure.

This demonstrates that the AI-agent can convert the numerical dashboard information into a readable maintenance summary. Instead of only showing individual numbers, the agent explains what the values mean in relation to the warning thresholds.

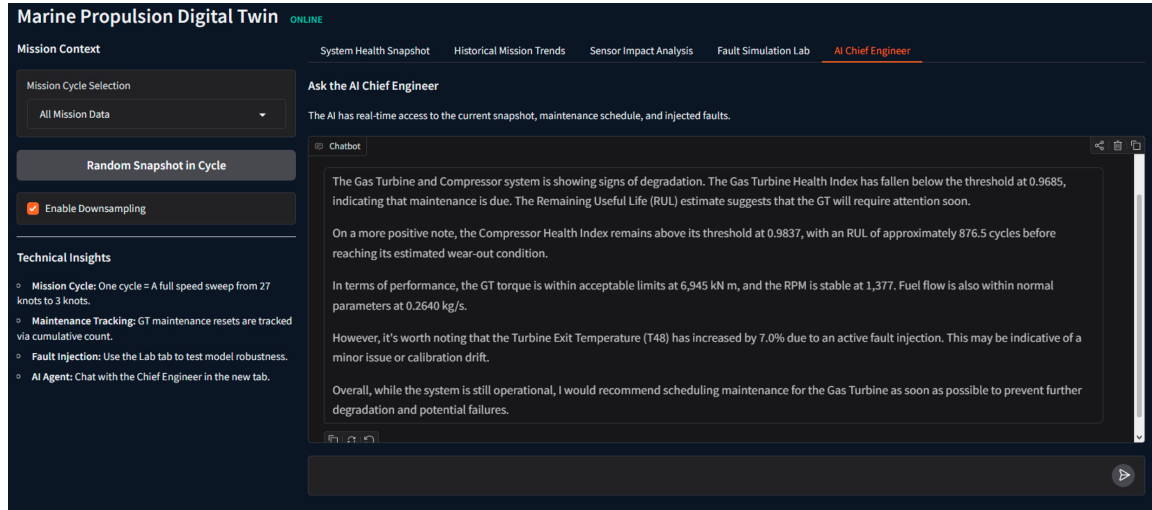


Figure 13: AI-agent response during a degraded condition with active turbine warning and injected temperature fault.

Figure 13 shows an example where the AI-agent responds during a degraded condition. The agent identifies that the gas turbine health index has fallen below the threshold and that maintenance is due. It also reports that the compressor remains above its threshold and that the turbine exit temperature has increased due to an active fault injection.

This shows the main benefit of the AI-agent. It combines several pieces of digital twin context into one explanation: component health, warning status, RUL estimate, sensor readings, and injected fault information. The agent then gives a maintenance-oriented recommendation, such as scheduling turbine maintenance to prevent further degradation.

The AI-agent uses the current dashboard state as context. This includes the predicted compressor and turbine health values, maintenance threshold status, estimated RUL, active fault injections, and selected sensor readings. The output is therefore not a general answer from the language model alone, but a response based on the digital twin state.

However, the AI-agent should be interpreted as a decision-support tool, not as a verified fault diagnosis system. The root cause information is based on sensor deviations and injected fault labels, not physical inspection. For this reason, the AI-agent can suggest possible causes and maintenance actions, but the final decision should still be made by an engineer using the dashboard results, model performance, and domain knowledge together.

7 Conclusion

This project developed a machine-learning-based digital twin for predictive maintenance of a marine gas turbine propulsion system. The dataset was prepared and used to train three supervised regression models: Random Forest, XGBoost, and SVM. The models were trained to predict the compressor and turbine decay state coefficients, which were used as health indicators for the propulsion system.

The model comparison showed that SVM gave the best overall prediction performance for both compressor and turbine health. It achieved the lowest test MAE and the highest test R^2 for both target variables. Based on these results, the SVM models were selected as the prediction layer in the digital twin.

The digital twin used the predicted decay coefficients for health monitoring, maintenance warning, maintenance event detection, and remaining useful life estimation. The dashboard made it possible to inspect current component health, sensor values, warning status, and estimated maintenance needs in a structured way. The AI-agent added an extra layer of user support by explaining the digital twin outputs in natural language and giving maintenance-oriented recommendations based on the current system state.

The final system demonstrates how machine learning, digital twin logic, dashboard visualization, and an AI-agent can be combined into a predictive maintenance tool. However, the project also has limitations. The dataset is based on steady-state operating data, the RUL estimation uses a simplified linear extrapolation, and the system has not been physically validated on a real vessel. Future work should include more operational data, testing with real sensor faults and maintenance events, improved RUL methods, and validation against physical gas turbine behaviour.

Bibliography

- [1] Leo Breiman. Random Forests. In: Machine Learning 45.1 (2001), pp. 5–32. DOI: 10.1023/A:1010933404324.
- [2] Tom B. Brown et al. Language Models are Few-Shot Learners. In: Advances in Neural Information Processing Systems 33 (2020), pp. 1877–1901.
- [3] Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. In: 2016, pp. 785–794. DOI: 10.1145/2939672.2939785.
- [4] Corinna Cortes and Vladimir Vapnik. Support-Vector Networks. In: Machine Learning 20.3 (1995), pp. 273–297. DOI: 10.1007/BF00994018.
- [5] Gradio. Gradio API Documentation. Accessed 2026-04-01. 2026. URL: <https://www.gradio.app/docs>.
- [6] Agus Hasan and Pierluigi Salvo Rossi. A Unified Sensor and Actuator Fault Diagnosis in Digital Twins for Remote Operations. In: Mechanical Systems and Signal Processing 222 (2025), p. 111778. DOI: 10.1016/j.ymssp.2024.111778.
- [7] Agus Hasan et al. Predictive Digital Twins for Autonomous Surface Vessels. In: Ocean Engineering 288 (2023), p. 116046.
- [8] Trevor Hastie, Robert Tibshirani and Jerome Friedman. The Elements of Statistical Learning. 2nd ed. Springer, 2009.
- [9] Gareth James et al. An Introduction to Statistical Learning: with Applications in Python. 1st ed. Springer, 2023.
- [10] Max Kuhn and Kjell Johnson. Feature Engineering and Selection: A Practical Approach for Predictive Models. CRC Press, 2019.
- [11] Yaguo Lei et al. Applications of Machine Learning to Machine Fault Diagnosis: A Review and Roadmap. In: Mechanical Systems and Signal Processing 138 (2020), p. 106587. DOI: 10.1016/j.ymssp.2019.106587.
- [12] Andreas C. Müller and Sarah Guido. Introduction to Machine Learning with Python: A Guide for Data Scientists. O'Reilly Media, 2016.
- [13] Ollama. Ollama Documentation. Accessed 2026-04-01. 2026. URL: <https://ollama.com/>.
- [14] Richard S. Sutton and Andrew G. Barto. Reinforcement Learning: An Introduction. 2nd ed. MIT Press, 2018.
- [15] Wayne Xin Zhao et al. A Survey of Large Language Models. In: arXiv preprint arXiv:2303.18223 (2023).

Appendix